

Operating Systems: Assignment-5

Name: Suraj Shah

Admission Number: I24AI012

Q Write a Simulator which runs different CPU scheduling algorithms and gives different Utilization matrices viz.

- a. CPU Utilization
- b. Waiting time of each process and average waiting time
- c. Response time of each process and average response time
- d. Turn-around time of each process and average turn-around time

Your simulator will take only two arguments – 1) Scheduling Algorithm 2) No. of Processes. On the basis of these two arguments you need to randomly generate multiple internal parameters. Some of the parameters are

- 1) Arrival Time
- 2) CPU Burst or Execution time
- 3) Deadline

Scheduling algorithms which you need to simulate are

- 1) FCFS
- 2) RR
- 3) SJF
- 4) SRTN
- 5) Priority or additionally any other algorithm if you want to include.

User may want to see results for all the algorithms in the single run. In that case, you need to use the same data for all the algorithms. Some of the points to ponder

- 1) Use a random number generator which is near a perfect random number generator.
- 2) In SJF there is no need of Arrival time. Consider all the jobs' arrival at the start.
- 3) In RR, you may show your results for multiple quantum sizes.
- 4) Preemptive and Non-Preemptive results need to be shown for different algorithms.
- 5) Your example run will look like this.

Result

root# CPUSimulator

Hi! Welcome to CPU Scheduling Simulator. Please give me required parameters.

Scheduling algorithm – (1) FCFS (2) RR (3) SJF (4) SRTN (5) Priority (6) All

ALL

No. of Processes in the system : 50

Wait.... Generating Schedules...

DONE. Please check output file Output. tex for all the results.

You can produce whole output at STDOUT also but due to a large number of processes you need to use a file for output. You may also use different files for different purposes. You need to create a file process <studentid>_data.tex which contains randomly generated data for given no. of processes.

Additional features.

1) Gantt charts

2) Comparison table at the end for all the algorithms.

Code:

```
import random
import sys
from collections import deque
import copy

class Process:
    def __init__(self, pid, arrival, burst, priority, deadline):
        self.pid = pid
        self.arrival = arrival
        self.burst = burst
        self.remaining = burst
        self.priority = priority
        self.deadline = deadline

        self.start_time = None
        self.finish_time = None
        self.response = None
        self.waiting = 0
        self.turnaround = 0
```

```

def generate_processes(n, seed=42):
    random.seed(seed)
    procs = []
    for i in range(n):
        arrival = random.randint(0, 10)
        burst = random.randint(1, 10)
        priority = random.randint(1, 5)
        deadline = arrival + burst + random.randint(5, 15)
        procs.append(Process(i, arrival, burst, priority, deadline))
    return procs

```

```

def finalize_metrics(procs):
    for p in procs:
        p.turnaround = p.finish_time - p.arrival
        p.waiting = p.turnaround - p.burst

```

```

def gantt_chart(schedule):
    chart = ""
    for pid in schedule:
        chart += f"P{pid}"
    chart += "\n"
    return chart

```

```

def fcfs(procs):
    time = 0
    schedule = []
    procs.sort(key=lambda x: x.arrival)

    for p in procs:
        if time < p.arrival:
            time = p.arrival
        p.start_time = time
        p.response = p.start_time - p.arrival
        for _ in range(p.burst):
            schedule.append(p.pid)
            time += 1
        p.finish_time = time

    finalize_metrics(procs)
    return schedule

```

```

def sjf(procs):
    time = 0
    schedule = []
    done = 0
    n = len(procs)

    while done < n:
        ready = [p for p in procs if p.arrival <= time and p.finish_time is None]
        if not ready:
            time += 1
            continue

        p = min(ready, key=lambda x: x.burst)

        p.start_time = time
        p.response = p.start_time - p.arrival
        for _ in range(p.burst):
            schedule.append(p.pid)
            time += 1

        p.finish_time = time
        done += 1

    finalize_metrics(procs)
    return schedule

```

```

def srtn(procs):
    time = 0
    schedule = []
    n = len(procs)
    completed = 0

    while completed < n:
        ready = [p for p in procs if p.arrival <= time and p.remaining > 0]
        if not ready:
            time += 1
            continue

        p = min(ready, key=lambda x: x.remaining)

        if p.start_time is None:
            p.start_time = time

```

```

        p.response = time - p.arrival

    schedule.append(p.pid)
    p.remaining -= 1
    time += 1

    if p.remaining == 0:
        p.finish_time = time
        completed += 1

    finalize_metrics(procs)
    return schedule

def rr(procs, quantum=2):
    time = 0
    schedule = []
    queue = deque()

    procs.sort(key=lambda x: x.arrival)
    i = 0
    n = len(procs)
    completed = 0

    while completed < n:
        while i < n and procs[i].arrival <= time:
            queue.append(procs[i])
            i += 1

        if not queue:
            time += 1
            continue

        p = queue.popleft()

        if p.start_time is None:
            p.start_time = time
            p.response = time - p.arrival

        run = min(quantum, p.remaining)
        for _ in range(run):
            schedule.append(p.pid)
            time += 1

```

```
p.remaining -= 1
```

```
while i < n and procs[i].arrival <= time:  
    queue.append(procs[i])  
    i += 1
```

```
if p.remaining > 0:  
    queue.append(p)  
else:  
    p.finish_time = time  
    completed += 1
```

```
finalize_metrics(procs)  
return schedule
```

```
def priority_sched(procs):
```

```
    time = 0  
    schedule = []  
    done = 0  
    n = len(procs)
```

```
    while done < n:  
        ready = [p for p in procs if p.arrival <= time and p.finish_time is None]  
        if not ready:  
            time += 1  
            continue
```

```
        p = min(ready, key=lambda x: x.priority)
```

```
        p.start_time = time  
        p.response = time - p.arrival  
        for _ in range(p.burst):  
            schedule.append(p.pid)  
            time += 1
```

```
        p.finish_time = time  
        done += 1
```

```
    finalize_metrics(procs)  
    return schedule
```

```
def write_output(filename, procs, schedule, name):
```

```

with open(filename, "a") as f:
    f.write(f"\n=== {name} ===\n")
    f.write(gantt_chart(schedule))

total_wait = total_resp = total_turn = 0

for p in procs:
    f.write(
        f"P{p.pid}: WT={p.waiting}, RT={p.response}, TAT={p.turnaround}\n"
    )
    total_wait += p.waiting
    total_resp += p.response
    total_turn += p.turnaround

n = len(procs)
f.write(f"Average WT: {total_wait/n:.2f}\n")
f.write(f"Average RT: {total_resp/n:.2f}\n")
f.write(f"Average TAT: {total_turn/n:.2f}\n")

def run_sim(algo, n):
    base = generate_processes(n)

    with open("process_data.tex", "w") as f:
        f.write("Generated process data:\n")
        for p in base:
            f.write(
                f"P{p.pid}: A={p.arrival}, B={p.burst}, Pr={p.priority}, D={p.deadline}\n"
            )

    algos = {
        "1": ("FCFS", fcfs),
        "2": ("RR", rr),
        "3": ("SJF", sjf),
        "4": ("SRTN", srt_n),
        "5": ("PRIORITY", priority_sched),
    }

    open("output.tex", "w").close()

    if algo == "6": # ALL
        for key, (name, fn) in algos.items():
            procs = copy.deepcopy(base)

```

```

        schedule = fn(procs) if key != "2" else fn(procs, 3)
        write_output("output.tex", procs, schedule, name)
    else:
        name, fn = algos[algo]
        procs = copy.deepcopy(base)
        schedule = fn(procs)
        write_output("output.tex", procs, schedule, name)

    print("DONE. Check output.tex")

if __name__ == "__main__":
    print("Hi! Welcome to CPU Scheduling Simulator.")
    print("(1) FCFS (2) RR (3) SJF (4) SRTN (5) Priority (6) All")

    algo = input("Scheduling algorithm: ")
    n = int(input("No. of processes: "))

    run_sim(algo, n)

```

Output:

```

ubuntu@ubuntu:~$ python3 cpu_simulator.py
Hi ! Welcome to CPU Scheduling Simulator.
(1) FCFS (2) RR (3) SJF (4) SRTN (5) Priority (6) All
Scheduling algorithm: 6
No. of processes: 10
DONE. Check output.tex
ubuntu@ubuntu:~$ cat process_data.tex
Generated process data:
P0: A=10, B=2, Pr=1, D=21
P1: A=3, B=4, Pr=2, D=13
P2: A=10, B=9, Pr=1, D=33
P3: A=6, B=1, Pr=1, D=13
P4: A=3, B=4, Pr=5, D=21
P5: A=0, B=9, Pr=2, D=24
P6: A=8, B=7, Pr=2, D=27
P7: A=9, B=5, Pr=1, D=21
P8: A=6, B=6, Pr=3, D=19
P9: A=3, B=6, Pr=1, D=15

```



```
ubuntu@ubuntu:~$ cat output.tex
```

```
=== FCFS ===
```

```
|P5|P5|P5|P5|P5|P5|P5|P5|P5|P1|P1|P1|P1|P4|P4|P4|P4|P9|P9|P9|P9|P9|P9|P3|P8|P8|P8|P8|P8|P8|P6|P6|P6|P6|P6|P6|P7|P7|P7|P7|P7|P0|P0|P2|P2|P2|P2|P2|P2|P2|P2|P2|
P5: WT=0, RT=0, TAT=9
P1: WT=6, RT=6, TAT=10
P4: WT=10, RT=10, TAT=14
P9: WT=14, RT=14, TAT=20
P3: WT=17, RT=17, TAT=18
P8: WT=18, RT=18, TAT=24
P6: WT=22, RT=22, TAT=29
P7: WT=28, RT=28, TAT=33
P0: WT=32, RT=32, TAT=34
P2: WT=34, RT=34, TAT=43
Average WT: 18.10
Average RT: 18.10
Average TAT: 23.40
```

```
=== RR ===
```

```
|P5|P5|P5|P1|P1|P1|P4|P4|P4|P9|P9|P9|P5|P5|P5|P3|P8|P8|P8|P1|P6|P6|P6|P7|P7|P7|P7|P4|P0|P0|P2|P2|P2|P9|P9|P9|P5|P5|P5|P8|P8|P8|P6|P6|P6|P7|P7|P2|P2|P2|P6|P2|P2|P2|
P5: WT=29, RT=0, TAT=38
P1: WT=13, RT=0, TAT=17
P4: WT=20, RT=3, TAT=24
P9: WT=26, RT=6, TAT=32
P3: WT=9, RT=9, TAT=10
P8: WT=29, RT=10, TAT=35
P6: WT=35, RT=12, TAT=42
P7: WT=32, RT=14, TAT=37
P0: WT=17, RT=17, TAT=19
P2: WT=34, RT=19, TAT=43
Average WT: 24.40
Average RT: 9.00
Average TAT: 29.70
```

```
=== SJF ===
```

```
|P5|P5|P5|P5|P5|P5|P5|P5|P5|P3|P0|P0|P1|P1|P1|P1|P4|P4|P4|P4|P7|P7|P7|P7|P7|P8|P8|P8|P8|P8|P8|P9|P9|P9|P9|P9|P9|P6|P6|P6|P6|P6|P6|P6|P2|P2|P2|P2|P2|P2|P2|P2|
P0: WT=0, RT=0, TAT=2
P1: WT=9, RT=9, TAT=13
P2: WT=34, RT=34, TAT=43
P3: WT=3, RT=3, TAT=4
P4: WT=13, RT=13, TAT=17
P5: WT=0, RT=0, TAT=9
P6: WT=29, RT=29, TAT=36
P7: WT=11, RT=11, TAT=16
P8: WT=19, RT=19, TAT=25
P9: WT=28, RT=28, TAT=34
Average WT: 14.60
Average RT: 14.60
Average TAT: 19.90
```

```
=== SRTN ===
```

```
|P5|P5|P5|P1|P1|P1|P1|P3|P4|P4|P0|P0|P4|P4|P7|P7|P7|P7|P7|P5|P5|P5|P5|P5|P5|P8|P8|P8|P8|P8|P8|P9|P9|P9|P9|P9|P9|P6|P6|P6|P6|P6|P6|P6|P6|P6|P6|P2|P2|P2|P2|P2|P2|P2|P2|
P0: WT=0, RT=0, TAT=2
P1: WT=0, RT=0, TAT=4
P2: WT=34, RT=34, TAT=43
P3: WT=1, RT=1, TAT=2
P4: WT=7, RT=5, TAT=11
P5: WT=16, RT=0, TAT=25
P6: WT=29, RT=29, TAT=36
P7: WT=5, RT=5, TAT=10
P8: WT=19, RT=19, TAT=25
P9: WT=28, RT=28, TAT=34
Average WT: 13.90
Average RT: 12.10
Average TAT: 19.20
```

```
=== PRIORITY ===
```

```
|P5|P5|P5|P5|P5|P5|P5|P5|P5|P3|P0|P0|P2|P2|P2|P2|P2|P2|P2|P2|P7|P7|P7|P7|P7|P9|P9|P9|P9|P9|P9|P1|P1|P1|P1|P6|P6|P6|P6|P6|P6|P6|P8|P8|P8|P8|P8|P8|P4|P4|P4|P4|
P0: WT=0, RT=0, TAT=2
P1: WT=29, RT=29, TAT=33
P2: WT=2, RT=2, TAT=11
P3: WT=3, RT=3, TAT=4
P4: WT=46, RT=46, TAT=50
P5: WT=0, RT=0, TAT=9
P6: WT=28, RT=28, TAT=35
P7: WT=12, RT=12, TAT=17
P8: WT=37, RT=37, TAT=43
P9: WT=23, RT=23, TAT=29
Average WT: 18.00
Average RT: 18.00
Average TAT: 23.30
```